

Java OOP Notes

Access Modifiers · Interfaces · Inheritance · Abstract Classes · Static · Enums · Constructors

Contents

1. Access Control	4
2. Interfaces & Multiple Inheritance	5
Nested Interfaces	6
Interfaces Can Be Extended	7
Default Interface Methods (extension methods)	7
3. Inheritance	7
A Superclass Variable Can Reference a Subclass Object	8
Using super	8
A Second Use for super	9
Using final with Inheritance	9
Reference Tables	9
4. Abstract Classes	10
Abstract Class vs. Interface	11
5. Understanding static	12
Dynamic Method Dispatch	13
6. Packages	14
7. Overloading Methods	14
8. Returning Objects	15
9. Enumerations	15
Enum and Inheritance	16
values(), ordinal() and valueOf()	16
Enum and Constructors	16
Enum and Methods	16
10. The this Keyword, final Fields, and finalize()	16

The this Keyword	16
final Fields	17
The finalize() Method	17

11. Constructors, Objects, and new	17
---	-----------

Inheritance and Constructors	17
Classes and Objects	18
The new Keyword	18
A Closer Look at new	18
Parameters and Arguments	19

Appendix: Quick Reference (Added)	19
--	-----------

1. Access Control

How a member can be accessed is determined by the access modifier attached to its declaration. Usually, you will want to restrict access to the data members of a class—allowing access only through methods. Also, there will be times when you will want to define methods that are private to a class.

Java's access modifiers are public, private, and protected. Java also defines a default access level. protected applies only when inheritance is involved.

When no access modifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package.

	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World (diff pkg & not subclass)
public	+	+	+	+	+
protected	+	+	+	+	
no modifier	+	+	+		
private	+				

+ : accessible blank : not accessible

```
package packageOne;
public class Base
{
    protected void display(){
        System.out.println("in Base");
    }
}

package packageTwo;
public class Derived extends packageOne.Base{
    public void show(){
        new Base().display();    // this is not working
        new Derived().display(); // is working
        display();               // is working
    }
}
```

protected allows access from subclasses and from other classes in the same package. We can use a child class to use a protected member outside the package, but only a child class object can access it. That's why any Derived class instance can access the protected method in Base. The other line creates a Base instance (not a Derived instance!). Access to protected methods of that instance is only allowed from objects of the same package.

display(); → allowed, because the caller, an instance of Derived, has access to protected members and fields of its subclasses, even if they're in different packages.

new Derived().display(); → allowed, because you call the method on an instance of Derived and that instance has access to the protected methods of its subclasses.

new Base().display(); → not allowed, because the caller's (the this instance) class is not defined in the same package as the Base class, so this can't access the protected method. It doesn't matter that the current class

subclasses a class from that package — that backdoor is closed.

Remember: any time we talk about a subclass having access to a superclass member, we could be talking about the subclass inheriting the member, not simply accessing the member through a reference to an instance of the superclass.

```
class C
    protected member;

// in a different package
class S extends C
    obj.member;    // only allowed if type of obj is S or a subclass of S
```

The motivation is probably as follows. If obj is an S, class S has sufficient knowledge of its internals, has the right to manipulate its members, and can do so safely. If obj is not an S, it's probably another subclass S2 of C, which S has no idea of. S2 may not even have existed when S was written. For S to manipulate S2's protected internals is dangerous — S2 would have no way of reasoning about who tampers with its state.

Now if obj is D, and D extends S, is it dangerous for S to access obj.member? Not really. How S uses member is shared knowledge of S and all its subclasses, including D. S as the superclass has the right to define behaviour, and D as the subclass has the obligation to accept and conform.

NOTE: For easier understanding, the rule can be simplified to require obj's (static) type to be exactly S. It's unusual for subclass D to appear in S; even if it happens that the static type of obj is D, the simplified rule handles it via upcasting: ((S)obj).member

2. Interfaces & Multiple Inheritance

Multiple inheritance is not available in Java. (If the same function exists in 2 classes, Java will skip it — hence no multiple inheritance.) Instead we have Java interfaces; they have abstract functions (no body).

An interface is like a class, but not completely — it is like an abstract class. By default, functions are public and abstract in an interface. Variables are final and static by default in an interface.

Interfaces specify only what the class is doing, not how it is doing it. The problem with multiple inheritance is that two classes may define different ways of doing the same thing, and the subclass can't choose which one to pick.

NOTE: Key difference between a class and an interface: a class can maintain state information (especially through instance variables), but an interface cannot.

Using an interface, you can specify a set of methods that can be implemented by one or more classes. Although similar to abstract classes, interfaces have an extra capability: a class can implement more than one interface, whereas a class can only inherit a single superclass (abstract or otherwise).

Using the keyword interface, you can fully abstract a class' interface from its implementation — you specify what a class must do, but not how it does it. Interfaces are syntactically similar to classes, but lack instance variables, and their methods are generally declared without a body. By providing the interface keyword, Java allows you to fully utilize the “one interface, multiple methods” aspect of polymorphism.

NOTE: Interfaces are designed to support dynamic method resolution at run time. Normally, for a method to be called from one class to another, both classes need to be present at compile time so the compiler can check

that method signatures are compatible — a static, non-extensible classing environment. Interfaces disconnect the definition of methods from the inheritance hierarchy: since interfaces are in a different hierarchy from classes, unrelated classes can implement the same interface. This is where the real power of interfaces is realized.

Beginning with JDK 8, it is possible to add a default implementation to an interface method. Thus it is now possible for an interface to specify some behavior. However, default methods are a special-use feature, and the original intent behind interfaces still remains.

Variables can be declared inside interface declarations.

NOTE: They are implicitly final and static, meaning they cannot be changed by the implementing class. They must also be initialized. All methods and variables in an interface are implicitly public.

NOTE: Methods that implement an interface must be declared public. The type signature of the implementing method must match exactly the signature specified in the interface definition.

It is both permissible and common for classes that implement interfaces to define additional members of their own.

NOTE: You can declare variables as object references that use an interface rather than a class type (similar to using a superclass reference to access a subclass object). Any instance of any class that implements the declared interface can be referred to by such a variable. The correct version is looked up dynamically at run time, based on the actual instance being referred to. The calling code can dispatch through an interface without knowing anything about the “callee.”

NOTE: Dynamic lookup of a method at run time incurs significant overhead compared to normal method invocation, so be careful about using interfaces casually in performance-critical code.

Nested Interfaces

An interface can be declared a member of a class or another interface — a member interface or nested interface. A nested interface can be declared public, private, or protected. This differs from a top-level interface, which must be declared public or use the default access level.

```
// This class contains a member interface.
class A {
    // this is a nested interface
    public interface NestedIF {
        boolean isNotNegative(int x);
    }
}
// B implements the nested interface.
class B implements A.NestedIF {
    public boolean isNotNegative(int x) {
        return x < 0 ? false : true;
    }
}
class NestedIFDemo {
    public static void main(String args[]) {
        // use a nested interface reference
        A.NestedIF nif = new B();
        if(nif.isNotNegative(10))
            System.out.println("10 is not negative");
    }
}
```

```

    if(nif.isNotNegative(-12))
        System.out.println("this won't be displayed");
    }
}

```

Interfaces Can Be Extended

One interface can inherit another using the keyword `extends` — same syntax as inheriting classes. Any class that implements an interface must implement all methods required by that interface, including any inherited from other interfaces.

Default Interface Methods (extension methods)

A primary motivation for default methods was to allow interfaces to be expanded without breaking existing code — e.g. if you add another body-less method to an interface, every implementing class would otherwise need to provide a body for it.

```

default String getString() {
    return "Default String";
}

```

For example, a class might implement two interfaces. If each provides default methods, some behavior is inherited from both.

- In all cases, a class implementation takes priority over an interface default implementation.
- If a class implements two interfaces that both have the same default method, but the class does not override it, an error results.
- If one interface inherits another, and both define a common default method, the inheriting interface's version takes precedence.

NOTE: Static interface methods are not inherited by either an implementing class or a sub-interface, so static interface methods must have a body — they cannot be abstract.

NOTE: When overriding methods, the access modifier should be the same or less restrictive, i.e. if the parent class method was protected, the override must be either protected or public.

3. Inheritance

To inherit a class, you incorporate the definition of one class into another using the `extends` keyword:

```

class subclass-name extends superclass-name {
    // body of class
}

```

You can only specify one superclass for any subclass you create — Java does not support inheritance of multiple superclasses into a single subclass. You can create a hierarchy of inheritance in which a subclass becomes a superclass of another subclass; however, no class can be a superclass of itself.

Although a subclass includes all members of its superclass, it cannot access members of the superclass that have been declared private.

A Superclass Variable Can Reference a Subclass Object

It is the type of the reference variable — not the type of the object it refers to — that determines what members can be accessed. When a reference to a subclass object is assigned to a superclass reference variable, you only have access to the parts of the object defined by the superclass.

```
plainbox      = weightbox;  
(superclass)  (subclass)
```

```
SUPERCLASS ref = new SUBCLASS(); // ref can only access methods available in SUPERCLASS
```

Using super

Whenever a subclass needs to refer to its immediate superclass, it can do so using the keyword `super`. `super` has two general forms: the first calls the superclass' constructor; the second accesses a member of the superclass that has been hidden by a member of the subclass.

```
BoxWeight(double w, double h, double d, double m) {  
    super(w, h, d); // call superclass constructor  
    weight = m;  
}
```

Here, `BoxWeight()` calls `super()` with arguments `w`, `h`, and `d`, causing the `Box` constructor to be called, which initializes width, height, and depth. `BoxWeight` no longer initializes these itself — it only initializes weight, its own unique value. This leaves `Box` free to make these values private if desired.

`super()` always refers to the superclass immediately above the calling class — true even in a multileveled hierarchy.

```
class Box {  
    private double width;  
    private double height;  
    private double depth;  
  
    // construct clone of an object  
    Box(Box ob) { // pass object to constructor  
        width = ob.width;  
        height = ob.height;  
        depth = ob.depth;  
    }  
}  
  
class BoxWeight extends Box {  
    double weight; // weight of box  
  
    // construct clone of an object  
    BoxWeight(BoxWeight ob) { // pass object to constructor  
        super(ob);  
        weight = ob.weight;  
    }  
}
```

NOTE: `super()` is passed an object of type `BoxWeight` — not of type `Box` — yet this still invokes the constructor `Box(Box ob)`. A superclass variable can be used to reference any object derived from that class, so a `BoxWeight` object can be passed to the `Box` constructor; of course, `Box` only has knowledge of its own members.

A Second Use for super

The second form of `super` acts somewhat like this, except it always refers to the superclass of the subclass in which it is used:

```
super.member
```

Here, `member` can be a method or an instance variable. This form is most applicable when member names of a subclass hide members of the same name in the superclass.

`super()` always refers to the constructor in the closest superclass. In a class hierarchy, if a superclass constructor requires parameters, all subclasses must pass those parameters “up the line,” whether or not the subclass needs parameters of its own.

Constructors complete their execution in order of derivation, because a superclass has no knowledge of any subclass — any initialization it needs is separate from, and possibly a prerequisite to, initialization performed by the subclass. Therefore it must complete execution first.

NOTE: If `super()` is not used in a subclass' constructor, the default (parameterless) constructor of each superclass will be executed.

Using final with Inheritance

The keyword `final` has three uses:

1. It can be used to create the equivalent of a named constant.
2. **Using final to prevent overriding** — to disallow a method from being overridden, specify `final` as a modifier at the start of its declaration. Methods declared `final` cannot be overridden.

Final methods can sometimes provide a performance enhancement: the compiler is free to inline calls to them because it “knows” they won't be overridden by a subclass — eliminating the overhead of a method call. Inlining is an option only with final methods. Normally, Java resolves method calls dynamically at run time (“late binding”); but since final methods cannot be overridden, a call to one can be resolved at compile time (“early binding”).

3. **Using final to prevent inheritance** — to prevent a class from being inherited, precede the class declaration with `final`.

NOTE: Declaring a class `final` implicitly declares all of its methods `final` too. It is illegal to declare a class both `abstract` and `final`, since an abstract class is incomplete by itself and relies on its subclasses to provide complete implementations.

NOTE: Although static methods can be inherited, there is no point overriding them in child classes, because the method in the parent class will always run no matter which object calls it. That is also why static interface methods cannot be inherited — they always run from the parent interface, and that is why static interface methods must have a body.

NOTE: Polymorphism does not apply to instance variables.

Reference Tables

The two tables below summarise Abstraction vs. Encapsulation, and the relative restrictiveness of the access modifiers.

Abstraction	Encapsulation
Abstraction is a feature of OOPs that hides the unnecessary detail but shows the essential information.	Encapsulation is also a feature of OOPs. It hides the code and data into a single entity or unit so that the data can be protected from the outside world.
It solves an issue at the design level.	Encapsulation solves an issue at implementation level.
It focuses on the external lookout.	It focuses on internal working.
It can be implemented using abstract classes and interfaces .	It can be implemented by using the access modifiers (private, public, protected).
It is the process of gaining information.	It is the process of containing the information.
In abstraction, we use abstract classes and interfaces to hide the code complexities.	We use the getters and setters methods to hide the data.
The objects are encapsulated that helps to perform abstraction.	The object need not to abstract that result in encapsulation.

Abstraction vs. Encapsulation

Most Restrictive ← → Least Restrictive				
Access Modifiers ->	private	Default/no-access	protected	public
Inside class	Y	Y	Y	Y
Same Package Class	N	Y	Y	Y
Same Package Sub-Class	N	Y	Y	Y
Other Package Class	N	N	N	Y
Other Package Sub-Class	N	N	Y	Y

Same rules apply for inner classes too, they are also treated as outer class properties

Access modifier restrictiveness, most to least restrictive

4. Abstract Classes

Sometimes you want to create a superclass that only defines a generalized form to be shared by all its subclasses, leaving it to each subclass to fill in the details. Such a class determines the nature of the methods

that subclasses must implement.

You may have methods that must be overridden by the subclass in order for the subclass to have any meaning. In this case, you want to ensure a subclass does indeed override all necessary methods. Java's solution is the abstract method:

```
abstract type name(parameter-list);
```

These methods are sometimes called the subclass's responsibility, because they have no implementation specified in the superclass. A subclass must override them — it cannot simply use the version defined in the superclass.

Any class that contains one or more abstract methods must also be declared abstract.

- There can be no objects of an abstract class.
- You cannot declare abstract constructors, or abstract static methods.
- You can declare static methods in an abstract class.

Because there can be no objects of an abstract class, if abstract static methods were allowed to be called, it would mean calling an empty (abstract) method through the class name, since it is static.

Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be declared abstract itself. Abstract classes can include as much implementation as they see fit — there can be concrete methods (methods with a body) in an abstract class.

Although abstract classes cannot be used to instantiate objects, they can be used to create object references, because Java's approach to run-time polymorphism is implemented through the use of superclass references.

NOTE: A public constructor on an abstract class doesn't make much sense, since you can't instantiate an abstract class directly (it can only be instantiated through a derived, non-abstract type).

Abstract Class vs. Interface

Aspect	Interface	Abstract Class
Type of methods	Only abstract methods	Abstract and non-abstract; from Java 8, default and static methods too
Final variables	Implicitly final	May contain non-final variables
Type of variables	Only static and final	final, non-final, static and non-static
Implementation	Can't provide the implementation of an abstract class	Can provide the implementation of an interface
Keyword	implements	extends
Multiple implementation	Can extend another interface only	Can extend one class and implement multiple interfaces
Accessibility of members	Public by default	Can have private, protected, etc.

5. Understanding static

When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object. Both methods and variables can be declared static. The most common example is `main()`, which is static because it must be called before any objects exist. A static method belongs to the class, not to the object.

A static method can access only static data — it cannot access non-static data (instance variables). A non-static member belongs to an instance and is meaningless without resolving which instance you mean. In a static context there is no instance, which is why you can't access a non-static member without explicitly mentioning an object reference. In fact, you can access a non-static member in a static context by specifying the object reference explicitly:

```
public class Human {  
  
    String message = "Hello World";  
  
    public static void display(Human human){  
        System.out.println(human.message);  
    }  
  
    public static void main(String[] args) {  
        Human kunal = new Human();  
        kunal.message = "Kunal's message";  
        Human.display(kunal);  
    }  
}
```

- A static method can call only other static methods; it cannot call a non-static method from it.
- A static method can be accessed directly by the class name and doesn't need any object.
- A static method cannot refer to “this” or “super” in any way.

If you need to do computation to initialize static variables, you can declare a static block that gets executed exactly once, when the class is first loaded.

```
// Demonstrate static variables, methods, and blocks.  
class UseStatic {  
    static int a = 3;  
    static int b;  
    static void meth(int x) {  
        System.out.println("x = " + x);  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
    }  
    static {  
        System.out.println("Static block initialized.");  
        b = a * 4;  
    }  
    public static void main(String args[]) {  
        meth(42);  
    }  
}
```

As soon as `UseStatic` is loaded, all static statements run: first `a` is set to 3, then the static block executes (printing a message and initializing `b` to `a*4`, or 12), then `main()` calls `meth()`, passing 42 to `x`. Output:

```
Static block initialized.  
x = 42  
a = 3  
b = 12
```

NOTE: main() is static since it must be accessible for an application to run, before any instantiation takes place.

NOTE: Only nested classes can be static. Static inner classes can have static variables.

You can't override inherited static methods, since in Java overriding takes place by resolving the type of object at run time (not compile time) and then calling the respective method. Static methods are class-level methods, so they are always resolved during compile time. Static interface methods are not inherited by either an implementing class or a sub-interface.

```
public class Static {  
  
    // class Test // ERROR  
    static class Test{  
        String name;  
  
        public Test(String name) {  
            this.name = name;  
        }  
    }  
  
    public static void main(String[] args) {  
        Test a = new Test("Kunal");  
        Test b = new Test("Rahul");  
  
        System.out.println(a.name); // Kunal  
        System.out.println(b.name); // Rahul  
    }  
}
```

The static keyword may modify the declaration of a member type C within the body of a non-inner class or interface T. Its effect is to declare that C is not an inner class — just as a static method of T has no current instance of T in its body, C also has no current instance of T, nor any lexically enclosing instances. Here, Test does not have any instance of its outer class Static; neither does main. But main and Test can each have instances of the other.

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, the method in the subclass is said to override the method in the superclass. When an overridden method is called from within its subclass, it always refers to the subclass's version; the superclass's version is hidden. Method overriding occurs only when the names and type signatures of the two methods are identical — otherwise the two methods are simply overloaded.

Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time — how Java implements run-time polymorphism. A superclass reference variable can refer to a subclass object. When an overridden method is called through a superclass reference, Java determines which version to execute based on the type of the object being referred to at the time the call occurs — made at run time. It is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.

NOTE: If B extends A, you can override a method in A through B while changing the return type of the method to B.

6. Packages

Packages are containers for classes, used to keep the class name space compartmentalized. For example, a package allows you to create a class named List and store it in your own package without concern that it will collide with some other class named List stored elsewhere. Packages are stored hierarchically and are explicitly imported into new class definitions. A package is both a naming and a visibility control mechanism.

```
package MyPackage;
```

Java uses file system directories to store packages: the .class files for classes declared part of MyPackage must be stored in a directory called MyPackage. Case is significant, and the directory name must match the package name exactly. A package hierarchy must be reflected in the file system, e.g. package java.awt.image; needs to be stored in java\awt\image on Windows. You cannot rename a package without renaming the directory in which the classes are stored.

How does the Java run-time system know where to look for packages you create?

1. By default, the Java run-time system uses the current working directory as its starting point.
2. You can specify a directory path or paths by setting the CLASSPATH environment variable.
3. You can use the -classpath option with java and javac to specify the path to your classes.

NOTE: When a package is imported, only items declared public within it are available to non-subclasses in the importing code.

7. Overloading Methods

In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. Overloaded methods may have different return types, but the return type alone is insufficient to distinguish two versions of a method. When Java encounters a call to an overloaded method, it executes the version whose parameters match the arguments used in the call. In some cases, Java's automatic type conversions can play a role in overload resolution.

```
class OverloadDemo {
    void test(double a){
        System.out.println("Inside test(double) a: " + a);
    }
}
class Overload {
    public static void main(String args[]) {
        OverloadDemo ob = new OverloadDemo();
        int i = 88;
        ob.test(i);           // this will invoke test(double)
        ob.test(123.2);       // this will invoke test(double)
    }
}
```

This version of OverloadDemo does not define test(int). When test() is called with an integer argument, no matching method is found directly, but Java can automatically convert an integer to a double, and this

conversion resolves the call — so `test(int)` is not found and Java elevates `i` to `double`, calling `test(double)`. If `test(int)` had been defined, it would have been called instead. Java employs automatic type conversions only if no exact match is found.

8. Returning Objects

```
// Returning an object.
class Test {
    int a;
    Test(int i) {
        a = i;
    }
    Test incrByTen() {
        Test temp = new Test(a+10);
        return temp;
    }
}
class RetOb {
    public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
    }
}
```

Output:

```
ob1.a: 2
ob2.a: 12
```

Each time `incrByTen()` is invoked, a new object is created, and a reference to it is returned to the calling routine. Since all objects are dynamically allocated using `new`, you don't need to worry about an object going out of scope because the method that created it terminates — the object continues to exist as long as there is a reference to it somewhere in the program. When there are no references to it, it will be reclaimed at the next garbage collection.

9. Enumerations

An enumeration is a list of named constants. In Java, an enumeration defines a class type. By making enumerations classes, the capabilities of the enumeration are greatly expanded. An enumeration is created using the `enum` keyword.

NOTE: Enum declaration can be done outside a class or inside a class, but not inside a method. We can declare `main()` inside an enum, so an enum can be invoked directly from the command prompt.

```
/* internally, enum Color is converted to:
class Color
{
    public static final Color Red = new Color();
    public static final Color Blue = new Color();
    public static final Color Green = new Color();
}
*/
```

Enum and Inheritance

- All enums implicitly extend `java.lang.Enum`. As a class can only extend one parent in Java, an enum cannot extend anything else.
- An enum cannot be a superclass.
- `toString()` is overridden in `java.lang.Enum`, returning the enum constant name.
- An enum can implement many interfaces.

Two enumeration constants can be compared for equality using the `==` relational operator.

`values()`, `ordinal()` and `valueOf()`

These methods are present inside `java.lang.Enum`.

- `values()` returns all values present inside the enum.
- `ordinal()` returns each enum constant's index (order matters in enums), just like an array index.
- `valueOf()` returns the enum constant of the specified string value, if it exists.

Enum and Constructors

- An enum can contain a constructor, executed separately for each enum constant when the enum class is loaded.
- We can't create enum objects explicitly, and hence can't invoke an enum constructor directly.
- The constructor cannot be public or protected — it must be private or default. If it could be public or protected, it would allow initializing more than one object, which is against the enum concept.

Enum and Methods

An enum can contain concrete methods only — i.e. no abstract methods.

You can compare an enumeration constant with any other object for equality using `equals()`, which overrides `Object's equals()`. Although `equals()` can compare an enumeration constant to any other object, the two will only be equal if they refer to the same constant within the same enumeration. Simply having matching ordinal values will not make `equals()` return true if the two constants are from different enumerations. Remember, you can also compare two enumeration references for equality using `==`.

10. The `this` Keyword, final Fields, and `finalize()`

The `this` Keyword

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the `this` keyword: `this` can be used inside any method to refer to the current object — that is, `this` is always a reference to the object on which the method was invoked.

final Fields

A field can be declared final, preventing its contents from being modified — essentially making it a constant. This means you must initialize a final field when it is declared. It is a common coding convention to choose all-uppercase identifiers for final fields:

```
final int FILE_OPEN = 2;
```

NOTE: final guarantees immutability only when instance variables are primitive types, not reference types. If an instance variable of a reference type has the final modifier, the reference itself will never change (it will always refer to the same object), but the value of the object it refers to can still change.

The finalize() Method

Sometimes an object needs to perform an action when it is destroyed. Java provides finalization for this: you can define specific actions that occur when an object is about to be reclaimed by the garbage collector. To add a finalizer to a class, define the finalize() method. The Java run time calls that method right before it recycles an object of that class.

```
protected void finalize( ) {  
    // finalization code here  
}
```

11. Constructors, Objects, and new

Once defined, a constructor is automatically called when the object is created, before the new operator completes. Constructors look a little strange because they have no return type, not even void — this is because the implicit return type of a class' constructor is the class type itself.

In the line `Box mybox1 = new Box();`, `new Box()` is calling the `Box()` constructor.

Inheritance and Constructors

In Java, the constructor of the base class with no arguments is automatically called in the derived class constructor. For example, the output of the following program is:

```
Base Class Constructor Called  
Derived Class Constructor Called
```

```
// filename: Main.java  
class Base {  
    Base() {  
        System.out.println("Base Class Constructor Called ");  
    }  
}  
  
class Derived extends Base {  
    Derived() {  
        System.out.println("Derived Class Constructor Called ");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Derived d = new Derived();  
    }  
}
```

```
}  
}
```

Every class has a default constructor, whether or not we declare it. If we inherit a class, the derived class must call its superclass constructor — done by default in the derived class. If the derived class does not have a default constructor, the JVM invokes its default constructor and calls the superclass constructor by default. Even if the derived class has a parameterised constructor, it still calls the default superclass constructor by default. If the superclass does not have a default constructor (only a parameterised one), the derived class constructor must explicitly call the parameterised superclass constructor.

Classes and Objects

A class is a template for an object, and an object is an instance of a class. A class creates a new data type that can be used to create objects. When you declare an object of a class, you are creating an instance of that class. A class is a logical construct; an object has physical reality (it occupies space in memory).

Objects are characterized by three essential properties: state, identity, and behavior. The state of an object is a value from its data type. The identity of an object distinguishes one object from another — it is useful to think of an object's identity as the place where its value is stored in memory. The behavior of an object is the effect of data-type operations.

The dot operator links the name of the object with the name of an instance variable. Although commonly called the dot operator, the formal Java specification categorizes the `.` as a separator.

The new Keyword

The `new` keyword dynamically allocates (allocates at run time) memory for an object and returns a reference to it — more or less the address in memory of the object allocated by `new`. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

```
Box mybox;           // declare reference to object  
mybox = new Box();   // allocate a Box object
```

The first line declares `mybox` as a reference to an object of type `Box`; at this point `mybox` does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to `mybox`. After that, you can use `mybox` as if it were a `Box` object — but in reality `mybox` simply holds the memory address of the actual `Box` object.

NOTE: The key to Java's safety is that you cannot manipulate references as you can actual pointers. You cannot cause an object reference to point to an arbitrary memory location or manipulate it like an integer.

A Closer Look at `new`

```
classname class-var = new classname ( );
```

Here, `class-var` is a variable of the class type being created; `classname` is the name of the class being instantiated. The class name followed by parentheses specifies the constructor for the class — a constructor defines what occurs when an object of a class is created.

You might wonder why you don't need `new` for integers or characters: Java's primitive types are not implemented as objects — they are implemented as “normal” variables, done in the interest of efficiency.

NOTE: new allocates memory for an object during run time.

```
Box b1 = new Box();  
Box b2 = b1;
```

b1 and b2 will both refer to the same object. The assignment of b1 to b2 did not allocate any memory or copy any part of the original object — it simply makes b2 refer to the same object as b1. Any changes made to the object through b2 will affect the object b1 refers to, since they are the same object. When you assign one object reference variable to another, you are not creating a copy of the object — only a copy of the reference.

Parameters and Arguments

```
int square(int i){  
    return i * i;  
}
```

A parameter is a variable defined by a method that receives a value when the method is called — in square(int i), i is a parameter. An argument is a value passed to a method when it is invoked — square(100) passes 100 as an argument; inside square(), the parameter i receives that value.

NOTE: Bus bus = new Bus(); — the lhs (reference, i.e. bus) is looked at by the compiler, and the rhs (object, i.e. new Bus()) is looked at by the JVM.

Appendix: Quick Reference (Added)

The table below wasn't in your original notes, but is added here because overriding and overloading are covered separately above and are commonly mixed up — having them side by side should help for quick revision.

Aspect	Overriding	Overloading
Where	Between superclass and subclass	Within the same class
Method signature	Must be identical (name + parameters)	Same name, different parameter list
Return type	Same, or a covariant (subclass) return type	Can differ freely
Binding	Dynamic (resolved at run time, by object type)	Static (resolved at compile time, by argument types)
Access modifier	Cannot be more restrictive than the superclass version	No restriction between overloaded versions
final / static methods	final and static methods cannot be overridden	final and static methods can still be overloaded